Apache Beam & Dataflow Flex Templates

From Programming Model to Packaged, Reusable Pipelines

This handbook explains the Apache Beam programming model, how Dataflow templates (Classic and Flex) turn a Beam pipeline into a reusable, parameterized artifact, and walks through building and running a Flex Template end-to-end, with Cloud Console navigation at every step.

Problem statement: A data engineering team has a working Beam pipeline, but every time someone needs to run it — a teammate, a scheduled Composer DAG, a different team entirely — they have to check out the source code, set up a matching Python environment, and run it by hand. Flex Templates solve this: package the pipeline once as a container image plus a small JSON spec, and anyone can launch it with just a job name and a few runtime parameters — no source code, no environment, no Python installation required on their end.

1. The Apache Beam Programming Model

Beam is a **unified programming model** for both batch and streaming data processing — you write one pipeline, and the exact same code can run against a bounded source (a file) or an unbounded one (a live stream), with only the source/sink and windowing changing, not the transform logic itself. Dataflow is Google's fully managed **runner** for Beam pipelines; the same pipeline code can also run on Apache Spark, Apache Flink, or locally via the DirectRunner.

1.1 Core Concepts

Concept	What it is
Pipeline	The top-level object representing your entire data processing job — a DAG of transforms.
PCollection	An immutable, distributed dataset flowing between transforms. Can be bounded (files) or unbounded (streams).
PTransform	An operation that takes one or more PCollections and produces one or more PCollections (e.g. Map, FlatMap, GroupByKey).
ParDo	The general-purpose, per-element transform — like a distributed 'apply this function to every element' (the basis of Map/Filter/FlatMap).
Runner	The execution engine a pipeline is submitted to: DirectRunner (local), DataflowRunner (managed GCP), SparkRunner, FlinkRunner.
Window	Divides an unbounded PCollection into finite chunks by time, so aggregations (like GroupByKey) have a defined boundary to operate over.

1.2 A Minimal Pipeline, Annotated

```
import apache_beam as beam
from apache_beam.options.pipeline_options import PipelineOptions

pipeline = beam.Pipeline(options=PipelineOptions())
(
    pipeline
    | 'ReadLines' >> beam.io.ReadFromText('gs://bucket/input.txt')   # PCollection of lines
    | 'ExtractWords' >> beam.FlatMap(lambda line: line.split())       # PCollection of words
    | 'CountWords' >> beam.combiners.Count.PerElement()               # PCollection of (word, count
)
    | 'FormatResult' >> beam.Map(lambda kv: f'{kv[0]}: {kv[1]}')      # PCollection of strings
    | 'WriteResult' >> beam.io.WriteToText('gs://bucket/output')
)
pipeline.run()
```

The pipe operator (|) chains PTransforms — each stage's output PCollection becomes the next stage's input. The string before >> (e.g. 'ExtractWords') is just a human-readable stage label, useful for reading the execution graph in the Dataflow console later.

1.3 Bounded vs. Unbounded, and Why Windowing Exists

A bounded PCollection (a file) has a known end — GroupByKey/Count naturally know when they've seen everything. An unbounded PCollection (a live topic) never ends, so an aggregation needs an artificial boundary — a **window** (e.g. "every 60 seconds") — to know when to actually emit a result. This is the same underlying reason streaming aggregations always need windowing while batch aggregations never do.

2. Dataflow Templates — Classic vs. Flex

Running python pipeline.py directly requires the runner (a person, or a scheduler) to have the source code, a matching Python environment, and all dependencies installed. A **template** packages a pipeline so it can be launched by name, with runtime parameters, by anyone with the right IAM permissions — no source code or environment needed on the launching side.

	Classic Templates	Flex Templates
How it's packaged	A JSON template staged in GCS, built via a special 'template' pipeline run	A Docker container image + a JSON spec file
Pipeline construction	Fixed at template creation time — parameters are just runtime *values* substituted into an otherwise-frozen graph	Happens at launch time, inside the container — parameters can influence pipeline *shape*, not just values
Custom dependencies	Limited — hard to include arbitrary system libraries	Full control — anything you can put in a Docker image
Current guidance	Legacy; being phased toward Flex for new work	Recommended for all new templated pipelines

The practical rule for the class: unless you're maintaining an existing Classic Template, build new templated pipelines as Flex Templates — they're strictly more capable and are Google's current recommended path.

3. Flex Template Architecture

A Flex Template has exactly two pieces:

Piece	What it is	Where it lives
Container image	A Docker image packaging your pipeline code, dependencies, and a launcher	Artifact Registry
Template spec (metadata.json / template.json)	A JSON file pointing at the container image, plus declared pipeline parameters (name, help text, validation regex)	Cloud Storage

Launching a Flex Template job tells Dataflow: "read this JSON spec, pull this container image, run its launcher with these parameter values." The launcher inside the container constructs the actual Beam pipeline graph at that moment, using the parameters you passed — this is why Flex Templates can do things Classic Templates can't (like choosing entirely different transforms based on a parameter, not just different parameter *values*).

Critical Python gotcha worth stating explicitly to the class: the pipeline construction script must call pipeline.run() and then exit — never wrap it in Beam's with beam.Pipeline(...) as p: context manager, since that calls the blocking wait_until_finish() on exit. A Flex Template's launcher process needs to exit immediately after submitting the job, not sit there waiting for the (potentially hours-long) job to complete.

4. Building and Running a Flex Template — Step by Step

4.1 Write the Pipeline, Using Runtime Value Providers

Parameters that should be settable at *launch* time (not baked in when the template is built) must be declared as ValueProvider arguments, not plain Python variables.

```
from apache_beam.options.pipeline_options import PipelineOptions

class TemplateOptions(PipelineOptions):
    @classmethod
    def _add_argparse_args(cls, parser):
        parser.add_value_provider_argument('--input_path', type=str)
        parser.add_value_provider_argument('--output_path', type=str)
```

4.2 Write requirements.txt and metadata.json

```
metadata.json:
{
  "name": "Word Count Flex Template",
  "description": "Reads a text file, counts word frequency, writes the result.",
  "parameters": [
    {"name": "input_path", "label": "Input file", "helpText": "GCS path to input text", "isOptional
": false},
    {"name": "output_path", "label": "Output prefix", "helpText": "GCS path prefix for output", "is
Optional": false}
  ]
}
```

4.3 Create an Artifact Registry Repository (One-Time per Project)

```
gcloud artifacts repositories create dataflow-templates --repository-format=docker --location=REGIO
N
```

4.4 Build the Template — Container + Spec, in One Command

No hand-written Dockerfile needed for a plain Python pipeline — --py-path plus --flex-template-base-image PYTHON3 tells Google's build process to containerize your files automatically.

```
gcloud dataflow flex-template build gs://BUCKET/templates/wordcount.json \
  --image-gcr-path REGION-docker.pkg.dev/PROJECT_ID/dataflow-templates/wordcount:latest \
  --sdk-language PYTHON \
  --flex-template-base-image PYTHON3 \
  --metadata-file metadata.json \
  --py-path . \
  --env FLEX_TEMPLATE_PYTHON_PY_FILE=pipeline.py \
  --env FLEX_TEMPLATE_PYTHON_REQUIREMENTS_FILE=requirements.txt
```

Check it in the console: Cloud Build > History → a new build should appear and complete (this is what's containerizing your pipeline). Artifact Registry > dataflow-templates repo → the pushed image. Cloud Storage > your bucket → templates/wordcount.json.

4.5 Run the Template

```
gcloud dataflow flex-template run wordcount-run-1 \
  --template-file-gcs-location gs://BUCKET/templates/wordcount.json \
  --region REGION \
```

```
  --parameters input_path=gs://BUCKET/input.txt \
  --parameters output_path=gs://BUCKET/output/result
```

Check it in the console: Dataflow > Jobs → wordcount-run-1 → the Job graph tab shows each labeled stage from pipeline.py (ReadLines → ExtractWords → CountWords → ...) as a visual DAG, with per-stage throughput once it's running.

5. When a Flex Template Is (and Isn't) the Right Tool

Use a Flex Template when...	Just run the pipeline directly when...
Non-engineers or a scheduler (Composer, Cloud Scheduler) need to launch it	Only the pipeline's own author ever runs it, during active development
The same pipeline logic is launched repeatedly with different parameters	You're iterating quickly and re-templating on every change would slow you down
You need to distribute a pipeline across teams without sharing source code/environment	The pipeline is a one-off exploratory job, never meant to be reused

This handbook's companion notebook: builds and runs exactly the word-count Flex Template shown in Section 4, end-to-end, against your own GCP project — including full cleanup — in roughly 15-20 minutes.